

UNIVERSIDADE FEDERAL DO PARANÁ
ESTRUTURA DE DADOS I (DS130)
PROF. HELCIO SOARES PADILHA JUNIOR

TRABALHO PRÁTICO I

DIÁRIO DE BORDO

**SISTEMA DE AGENDAMENTO DE CONSULTAS
EM CLÍNICA UNIVERSITÁRIA**

Aluna: ISABELLA PINHEIRO GALUSKI DA CRUZ

Repositório: <https://github.com/isabella18pgdc/agendamento-clinica>

ORGANIZAÇÃO DO TRABALHO

Fiz o trabalho sozinha, sou a única integrante do grupo.

SEMANA 1: PLANEJAMENTO, STRUCTS E PRIMEIRAS LISTAS

O que eu fiz:

Li o enunciado com calma e decidi quais listas usar. O histórico já tinha que ser lista com cabeçalho. Para o resto pensei em como cada dado se comporta: as salas são poucas e fixas (lista estática), os pacientes só entram e nunca saem (lista encadeada simples) e os agendamentos entram e saem toda hora (lista duplamente encadeada).

Também criei uma regra: cada paciente só pode ter uma consulta por dia. Como a remoção é por CPF + data, se a pessoa tivesse duas consultas no mesmo dia o sistema não ia saber qual apagar.

Nessa semana fiz as structs (Data, Paciente, Agendamento, Sala), a valida_data com ano bissexto, a lista estática de salas, a lista simples de pacientes com inserção em ordem alfabética, o cadastra_paciente e um menu simples só para ir testando. Também criei o repositório no GitHub.

Dificuldades e como resolvi:

- **O primeiro paciente "sumia".** Depois de cadastrar, a lista continuava vazia no main. O ponteiro do início estava sendo passado por valor, então a função mudava só uma cópia. Resolvi passando o endereço (NoPaciente **inicio) e chamando com &pacientes. Foi o que mais demorou para entender, desenhei as caixinhas e setas no caderno.
- **O fgets do nome era pulado.** Era o '\n' que o scanf do menu deixava no buffer. Reaproveitei a remove_quebra da lista de exercícios de funções e criei a limpa_buffer e a le_texto.
- **Ordem alfabética errada.** Com strcmp, "ana" (minúsculo) ia para depois de "Bruno", porque minúscula tem código ASCII maior. Fiz a compara_nomes, que compara letra por letra com toupper.
- **Structs definidas duas vezes.** Quando paciente.h e agendamento.h incluíram o lista.h deu erro de redefinição. Resolvi com #ifndef / #define / #endif.

SEMANA 2: AGENDAMENTOS, HISTÓRICO E TESTES

O que eu fiz:

Fiz a lista dupla de agendamentos (inserção ordenada, busca e remoção), o `cadastra_agendamento` com todas as validações, o `remove_agendamento` com confirmação antes de apagar, o histórico com cabeçalho, a consulta por CPF, a listagem por sala e a grade de horários livres. Depois juntei tudo no menu, coloquei a liberação de memória na saída e passei bastante tempo testando, tentando "quebrar" o programa: letra no lugar de número, data que não existe, CPF com ponto, desmarcar consulta que não existe etc. No final escrevi o README e este diário.

Dificuldades e como resolvi:

- **Segmentation fault ao desmarcar a última consulta.** Eu fazia `no->prox->ant = no->ant` sem ver se existia próximo. No último nó, `prox` é `NULL`. Coloquei `if (no->prox != NULL)`. E quando `ant` é `NULL` (primeiro nó), quem muda é o início da lista.
- **Agendamentos fora de ordem.** A primeira comparação olhava só o dia, e 05/12 aparecia antes de 10/11. A `compara_agendamentos` compara ano, depois mês, dia e hora.
- **Loop infinito no menu.** Digitando letra, o `scanf` não lia nada, a letra ficava no buffer e o menu repetia para sempre. A `le_inteiro` confere o retorno do `scanf`, limpa o buffer e devolve -1, que cai em "opção inválida".
- **Histórico mostrando lixo.** No começo o nó do histórico guardava um ponteiro para o nó do agendamento. Quando a consulta era desmarcada, o `free` apagava esse nó. Mudei para o histórico guardar uma cópia dos dados e o nome do paciente.

TRECHO 1: INSERÇÃO ORDENADA NA LISTA DE PACIENTES

Código-fonte (lista.c):

```
/* Insere o paciente ja na posicao certa da ordem alfabetica. */
/* Recebe ponteiro para ponteiro porque, quando insere no */
/* inicio, o primeiro no da lista muda. */
/* Devolve 1 se inseriu e 0 se faltou memoria */
/* Autoria: Isabella Pinheiro */
int insere_paciente_ordenado(NoPaciente **inicio, Paciente p)
{
    NoPaciente *novo;
    NoPaciente *atual;

    novo = (NoPaciente *)malloc(sizeof(NoPaciente));
    if (novo == NULL)
        return 0;

    novo->dados = p;
    novo->prox = NULL;

    /* Lista vazia ou nome vem antes do primeiro: entra no inicio */
    if (*inicio == NULL ||
        compara_nomes(p.nome, (*inicio)->dados.nome) < 0) {
        novo->prox = *inicio;
        *inicio = novo;
        return 1;
    }

    /* Anda ate o no que vai ficar ANTES do novo */
    atual = *inicio;
    while (atual->prox != NULL &&
        compara_nomes(atual->prox->dados.nome, p.nome) < 0)
        atual = atual->prox;

    novo->prox = atual->prox;
    atual->prox = novo;

    return 1;
}
```

Explicação:

Primeiro a função cria o nó com malloc. Depois vê se o paciente tem que ir para o começo: isso acontece quando a lista está vazia ou quando o nome dele vem antes do primeiro. Nesse caso o novo aponta para o antigo primeiro e vira o início (por isso o ponteiro para ponteiro). Se não for no começo, o while anda até o nó que fica ANTES da posição certa, o novo aponta para quem vinha depois e o anterior aponta para o novo. A ordem dessas duas linhas importa: se trocar, perde o resto da lista.

Saída:

Cadastrei na ordem maria souza, Ana Lima e Carlos Pereira, e a listagem saiu em ordem alfabética (e o "maria" minúsculo foi para o lugar certo):

Escolha uma opcao: 2

===== PACIENTES CADASTRADOS =====

```
1) Ana Lima | CPF: 98765432100 | Estudante - Enfermagem
2) Carlos Pereira | CPF: 11122233344 | Professor - Departamento de Informatica
3) maria souza | CPF: 12345678901 | Estudante - Analise e Desenvolvimento de Sistemas
Total: 3 paciente(s)
```

TRECHO 2: REMOÇÃO NA LISTA DUPLAMENTE ENCADEADA

Código-fonte (lista.c):

```
/* Acha a consulta pelo CPF + data e tira o no da lista dupla. */
/* Antes do free copia os dados para "removido" (historico). */
/* Como o no conhece o anterior (ant), nao precisa guardar o */
/* anterior enquanto percorre, como na lista simples. */
/* Devolve 1 se removeu e 0 se nao encontrou */
/* Autoria: Isabella Pinheiro */
int remove_no_agendamento(NoAgendamento **inicio, char cpf[], Data d,
                          Agendamento *removido)
{
    NoAgendamento *no;

    no = busca_agendamento(*inicio, cpf, d);
    if (no == NULL)
        return 0;

    *removido = no->dados;

    if (no->ant == NULL)          /* era o primeiro da lista */
        *inicio = no->prox;
    else
        no->ant->prox = no->prox;

    if (no->prox != NULL)        /* nao era o ultimo */
        no->prox->ant = no->ant;

    free(no);

    return 1;
}
```

Explicação:

A busca acha o nó pelo CPF e pela data. Como a lista é dupla, o nó já sabe quem é o anterior (no->ant), então não precisa ficar guardando o anterior durante a busca. Para tirar o nó, ele é "pulado": o anterior aponta para o próximo e o próximo aponta para o anterior. Se era o primeiro, muda o início da lista. Se era o último, não tem próximo para arrumar. Antes do free os dados são copiados para "removido", que vai para o histórico como CANCELADO.

Saída:

Desmarcando a consulta da maria souza no dia 10/11/2026:

```
Escolha uma opcao: 7

===== DESMARCAR CONSULTA =====
CPF do paciente: 12345678901
Data da consulta (dd/mm/aaaa): 10/11/2026

Consulta encontrada:
  10/11/2026 as 14:00 | Sala 2 (Odontologia) | maria souza
Confirma o cancelamento? (S/N): S
Consulta desmarcada com sucesso!
```

TRECHO 3: INSERÇÃO NO HISTÓRICO COM CABEÇALHO

Código-fonte (lista.c):

```
/* Registra AGENDADO ou CANCELADO no final do historico. */
/* Com o ponteiro "fim" do cabecalho nao precisa percorrer a */
/* lista para inserir no final. Tambem atualiza os contadores. */
/* Devolve 1 se inseriu e 0 se faltou memoria */
/* Autoria: Isabella Pinheiro */
int insere_historico(Cabecalho *h, char acao[], Agendamento a,
                    char nome[])
{
    NoHistorico *novo;

    novo = (NoHistorico *)malloc(sizeof(NoHistorico));
    if (novo == NULL)
        return 0;

    /* Guarda uma COPIA dos dados: o agendamento original pode */
    /* ser apagado depois, quando a consulta for desmarcada */
    novo->numero = h->qtd + 1;
    strcpy(novo->acao, acao);
    strcpy(novo->nome_paciente, nome);
    novo->dados = a;
    novo->prox = NULL;

    if (h->inicio == NULL)          /* historico vazio */
        h->inicio = novo;
    else
        h->fim->prox = novo;        /* liga depois do ultimo */

    h->fim = novo;
    h->qtd++;

    if (strcmp(acao, "AGENDADO") == 0)
        h->total_agendados++;
    else
        h->total_cancelados++;

    return 1;
}
```

Explicação:

O cabeçalho é uma struct separada que guarda o início, o fim e os contadores. Como ele tem o ponteiro fim, para colocar um registro no final não precisa percorrer a lista: liga o novo depois do último e atualiza o fim. Os contadores são atualizados na inserção, então o resumo do histórico (registros, agendados e cancelados) já fica pronto no cabeçalho.

Saída:

Histórico depois de 3 agendamentos e 1 cancelamento:

Escolha uma opção: 8

===== HISTORICO DE AGENDAMENTOS =====

Registros: 4 | Agendados: 3 | Cancelados: 1

#1 [AGENDADO] 10/11/2026 14:00 | Sala 2 (Odontologia)
Paciente: maria souza - CPF 12345678901
#2 [AGENDADO] 10/11/2026 09:00 | Sala 2 (Odontologia)
Paciente: Ana Lima - CPF 98765432100
#3 [AGENDADO] 05/11/2026 10:00 | Sala 2 (Odontologia)
Paciente: Carlos Pereira - CPF 11122233344
#4 [CANCELADO] 10/11/2026 14:00 | Sala 2 (Odontologia)
Paciente: maria souza - CPF 12345678901

TRECHO 4: COMPARAÇÃO DE DATA E HORA

Código-fonte (lista.c):

```
/* Diz qual agendamento vem primeiro no tempo: compara ano, */
/* depois mes, dia, hora e, se empatar, a sala */
/* Funciona igual ao strcmp: negativo, zero ou positivo */
/* Autoria: Isabella Pinheiro */
int compara_agendamentos(Agendamento a, Agendamento b)
{
    if (a.data.ano != b.data.ano)
        return a.data.ano - b.data.ano;

    if (a.data.mes != b.data.mes)
        return a.data.mes - b.data.mes;

    if (a.data.dia != b.data.dia)
        return a.data.dia - b.data.dia;

    if (a.hora != b.hora)
        return a.hora - b.hora;

    return a.sala - b.sala;
}
```

Explicação:

Funciona igual ao strcmp, só que para datas: devolve negativo se a consulta a vem antes da b, positivo se vem depois e zero se for igual. Começa pelo ano porque é o que mais "pesa"; só se o ano for igual olha o mês, e assim por diante. É ela que a `insere_agendamento_ordenado` usa para manter a lista em ordem cronológica.

Saída:

Agendei na sala 2: 10/11 às 14h, depois 10/11 às 9h e por último 05/11 às 10h. Mesmo assim a listagem saiu em ordem:

```
Escolha uma opcao: 5

===== AGENDAMENTOS POR SALA =====

Numero da sala: 2

Consultas da sala 2 (Odontologia):
  05/11/2026 as 10:00 | Sala 2 (Odontologia) | Carlos Pereira
  10/11/2026 as 09:00 | Sala 2 (Odontologia) | Ana Lima
  10/11/2026 as 14:00 | Sala 2 (Odontologia) | maria souza
Total: 3 consulta(s)
```

TRECHO 5: VERIFICAR SE UM HORÁRIO ESTÁ OCUPADO

Código-fonte (lista.c):

```
/* Verifica se ja tem consulta na sala, na data e na hora */
/* Devolve 1 se esta ocupado e 0 se esta livre */
/* Autoria: Isabella Pinheiro */
int horario_ocupado(NoAgendamento *inicio, int sala, Data d, int hora)
{
    NoAgendamento *atual;

    atual = inicio;
    while (atual != NULL) {
        if (atual->dados.sala == sala && atual->dados.hora == hora &&
            mesma_data(atual->dados.data, d))
            return 1;

        atual = atual->prox;
    }

    return 0;
}
```

Explicação:

Percorre a lista de agendamentos procurando uma consulta na mesma sala, mesma data e mesma hora. Se achar, o horário está ocupado. É usada para não deixar marcar duas consultas no mesmo horário e também para montar a grade de horários livres, onde é chamada uma vez para cada hora do dia.

Saída:

Grade da sala 2 no dia 10/11/2026 (com as consultas das 9h e das 14h):

```
Escolha uma opcao: 6

===== HORARIOS DISPONIVEIS =====

Numero da sala: 2
Data (dd/mm/aaaa): 10/11/2026

Horarios da sala 2 em 10/11/2026:
08:00  livre
09:00  OCUPADO
10:00  livre
11:00  livre
12:00  -- almoco --
13:00  livre
14:00  OCUPADO
15:00  livre
16:00  livre
17:00  livre
Horarios livres nesse dia: 7
```

TESTES REALIZADOS

CPF repetido	CPF ja cadastrado	OK
CPF com ponto e traco	CPF invalido	OK
Nomes com minuscula e maiuscula	ordem alfabetica certa	OK
Data 31/02/2026	data invalida	OK
Consulta as 12h ou as 19h	fora do atendimento	OK
Mesma sala, data e hora	horario ocupado	OK
Mesmo paciente 2x no mesmo dia	bloqueado (1 por dia)	OK
Desmarcar primeiro, meio e ultimo ..	remove sem erro	OK
Desmarcar consulta que nao existe ..	nao encontrada	OK
Letra no menu	opcao invalida, sem travar	OK
Historico apos agendar e cancelar ..	ordem e contadores certos	OK

CONSIDERAÇÕES FINAIS

O trabalho me ajudou a entender na prática a diferença entre os tipos de lista, que na aula parecia só teoria. Cada uma tem sua vantagem: o vetor é simples quando o tamanho é fixo, a lista simples resolve bem quando só se insere e busca, a dupla facilita muito a remoção e o cabeçalho deixa a inserção no final e os totais mais rápidos. A parte mais difícil foi ponteiro para ponteiro e os casos especiais (lista vazia, primeiro nó e último nó), que foram justamente onde apareceram os meus erros. Se tivesse mais tempo, eu gostaria de salvar os dados em arquivo para não perder tudo quando o programa fecha.